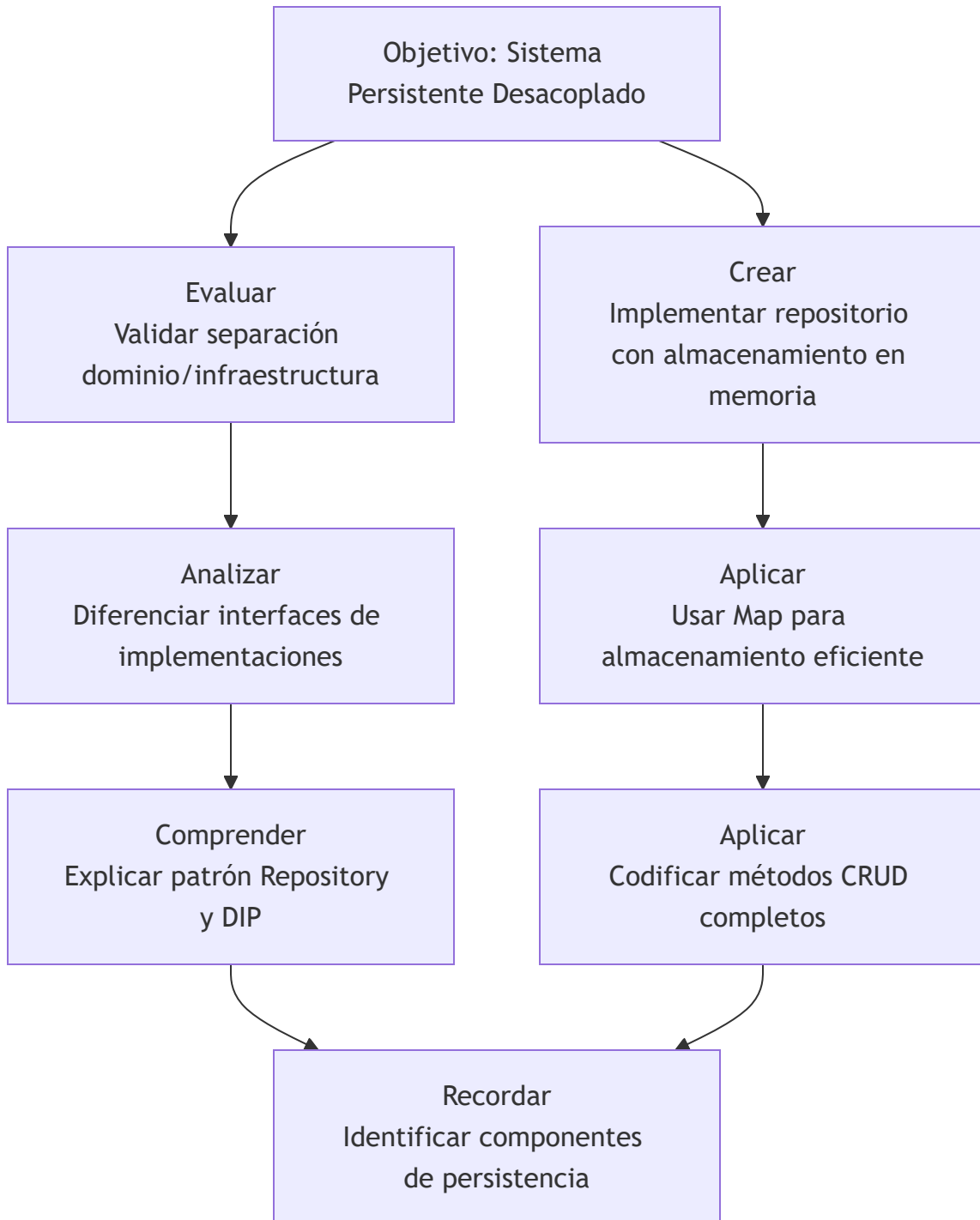


Guía 3: Capa de Persistencia - Patrón Repository con Arquitectura

Hexagonal

Objetivos de Aprendizaje

Pirámide de Objetivos



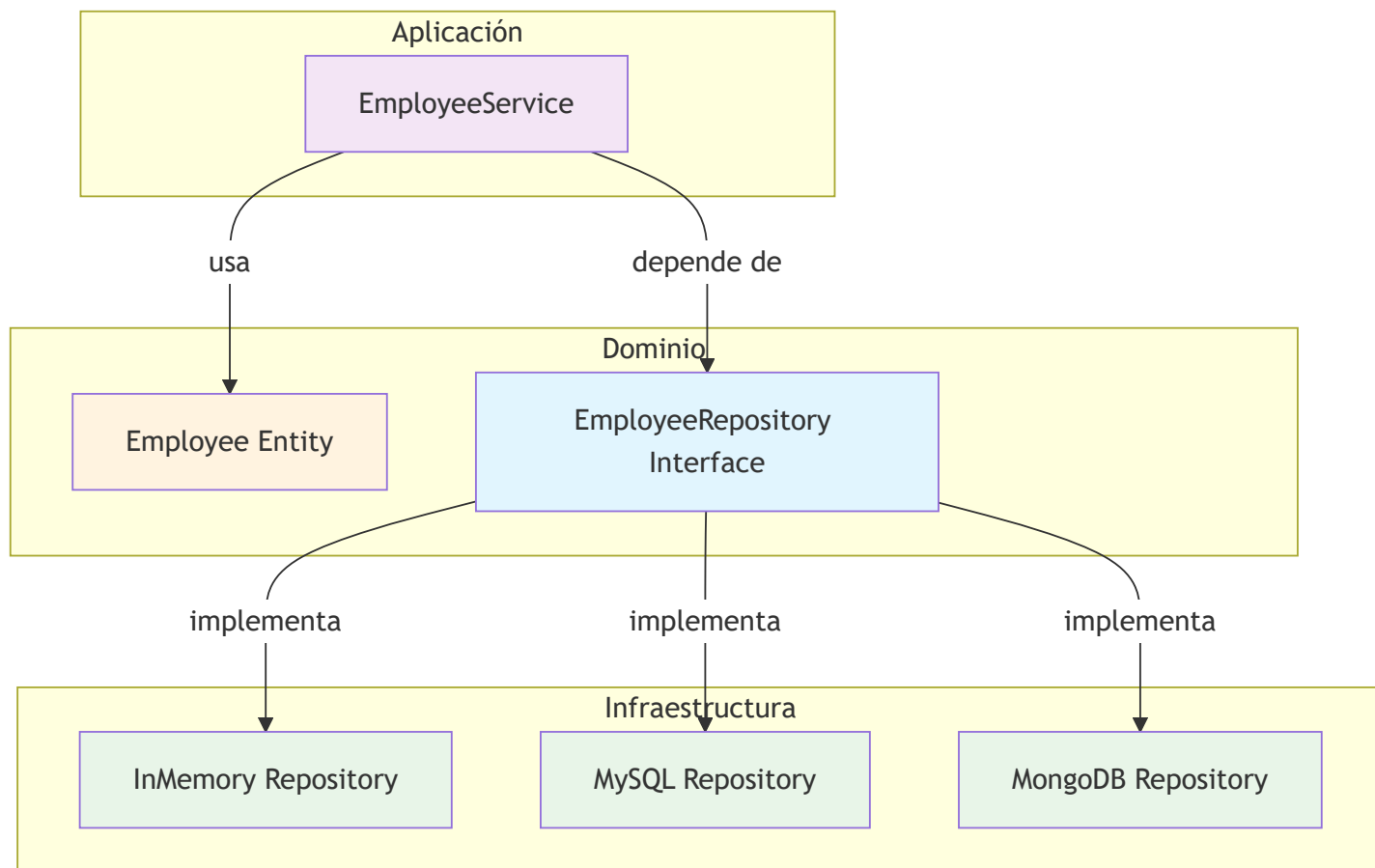
Objetivos Específicos

Nivel	Verbo	Objetivo	Evaluación
Recordar	Identificar	Componentes de la capa de persistencia	Listar interfaz y 3 implementaciones posibles
Comprender	Explicar	Principio de Inversión de Dependencias (DIP)	Diagrama que muestre dependencias correctas
Aplicar	Implementar	Interfaz EmployeeRepository con métodos CRUD	Código funcional en TypeScript
Aplicar	Crear	InMemoryEmployeeRepository usando Map	Implementación completa que pasa pruebas
Analizar	Diferenciar	Qué va en dominio vs infraestructura	Tabla con ejemplos correctamente clasificados
Evaluar	Probar	Todas las operaciones del repositorio	Script de pruebas ejecutándose exitosamente
Crear	Diseñar	Sistema que permite cambiar de MySQL a memoria	Demostración con cambio en una línea



1. ¿Qué es la Capa de Persistencia en Arquitectura Hexagonal?

Diagrama de Ubicación Arquitectónica



Definición y Propósito

Capa de Persistencia: Componente responsable de almacenar y recuperar datos, **implementando el patrón Repository** para abstraer los detalles técnicos del almacenamiento.

En Arquitectura Hexagonal:

- **Interfaz (Puerto):** En el **DOMINIO** - Define el "QUÉ" (contrato)
- **Implementación (Adaptador):** En **INFRAESTRUCTURA** - Define el "CÓMO" (detalles)

¿POR QUÉ esta separación?

// ❌ TRADICIONAL (Acoplado)

App → MySQLImplementation

// Cambiar a MongoDB = Reescribir toda la app

// ✅ HEXAGONAL (Desacoplado)

App → RepositoryInterface ← Memory/MySQL/MongoDB/File

// Cambiar a MongoDB = Cambiar UNA implementación

2. Implementación Paso a Paso

2.1 Paso 1: La Interfaz en el DOMINIO (Contrato)

```
// 📁 src/domain/employee.repository.ts
// 🎯 UBICACIÓN: DOMINIO - Define QUÉ necesita el negocio

/**
 * ⚠️ IMPORTANTE: Esta interfaz está en DOMINIO porque:
 * 1. Define CONTRATO sin detalles técnicos
 * 2. Es parte del LENGUAJE del negocio
 * 3. Permite al DOMINIO controlar sus necesidades
 * 4. Facilita testing (mockeable)
 */

import { Employee } from '../employee.entity.js';

export interface EmployeeRepository {
  // 📝 CRUD Básico
  save(employee: Employee): Promise<void>;
  findById(id: string): Promise<Employee | null>;
  findAll(): Promise<Employee[]>;
  update(employee: Employee): Promise<void>;
  delete(id: string): Promise<boolean>;

  // 🔍 Búsquedas específicas del negocio
  findByCedula(cedula: string): Promise<Employee | null>;
  findBySalaryRange(min: number, max: number): Promise<Employee[]>;

  // 📊 Operaciones de negocio
  count(): Promise<number>;
  existsByCedula(cedula: string): Promise<boolean>;
}
```

¿POR QUÉ en DOMINIO?

- ✅ **Control:** El dominio define sus necesidades
- ✅ **Independencia:** No sabe ni le importa MySQL/MongoDB/Memoria
- ✅ **Lenguaje Ubicuo:** `findByCedula()` es término de negocio
- ✅ **Testeable:** Se puede mockear para pruebas unitarias

2.2 Paso 2: Implementación en INFRAESTRUCTURA

(Memoria)

```
// 📁 src/infrastructure/repositories/in-memory.repository.ts
// 🌀 UBICACIÓN: INFRAESTRUCTURA - Implementa CÓMO funciona

/**
 * ⚠ IMPORTANTE: Esta clase está en INFRAESTRUCTURA porque:
 * 1. Contiene DETALLES TÉCNICOS (Map, console.log)
 * 2. Es un ADAPTADOR específico
 * 3. Puede cambiar sin afectar el dominio
 * 4. Implementa el contrato del dominio
 */

import type { EmployeeRepository } from '../../domain/employee.repository.js';
import { Employee, type EmployeeData } from '../../domain/employee.entity.js';

export class InMemoryEmployeeRepository implements EmployeeRepository {
  // 🔧 DETALLE TÉCNICO: Usamos Map para eficiencia O(1)
  private employees: Map<string, EmployeeData> = new Map();

  async save(employee: Employee): Promise<void> {
    const data = employee.toJSON();
    this.employees.set(data.id, data);
    console.log(`💾 Guardado en memoria: ${data.id}`);
  }

  async findById(id: string): Promise<Employee | null> {
    const data = this.employees.get(id);
    return data ? Employee.fromData(data) : null;
  }

  async findAll(): Promise<Employee[]> {
    const employees: Employee[] = [];
    for (const data of this.employees.values()) {
      employees.push(Employee.fromData(data));
    }
    return employees.sort((a, b) =>
      b.getCreatedAt().getTime() - a.getCreatedAt().getTime()
    );
  }

  async update(employee: Employee): Promise<void> {
    const id = employee.getId();
```



```

    if (!this.employees.has(id)) {
        throw new Error(`Empleado ${id} no encontrado`);
    }
    this.employees.set(id, employee.toJSON());
}

async delete(id: string): Promise<boolean> {
    return this.employees.delete(id);
}

async findByCedula(cedula: string): Promise<Employee | null> {
    for (const data of this.employees.values()) {
        if (data.cedula === cedula) {
            return Employee.fromData(data);
        }
    }
    return null;
}

async findBySalaryRange(min: number, max: number): Promise<Employee[]> {
    const employees: Employee[] = [];
    for (const data of this.employees.values()) {
        if (data.salario >= min && data.salario <= max) {
            employees.push(Employee.fromData(data));
        }
    }
    return employees.sort((a, b) => b.getSalario() - a.getSalario());
}

async count(): Promise<number> {
    return this.employees.size;
}

async existsByCedula(cedula: string): Promise<boolean> {
    for (const data of this.employees.values()) {
        if (data.cedula === cedula) return true;
    }
    return false;
}

// 🛠 Métodos de utilidad para testing
clear(): void {
    this.employees.clear();
}

```

```
}

async seed(): Promise<void> {
  const samples = [
    Employee.create('1234567890', 'Juan Pérez', 2500),
    Employee.create('0987654321', 'María García', 3200),
    Employee.create('5555555555', 'Carlos López', 1800)
  ];
  for (const emp of samples) await this.save(emp);
}
}
```

¿POR QUÉ en INFRAESTRUCTURA?

-  **Separación:** Detalles técnicos aparte de lógica de negocio
-  **Cambiable:** Puede reemplazarse por MySQL sin tocar dominio
-  **Responsabilidad Única:** Solo se encarga de persistencia en memoria
-  **Testeable Independiente:** Se prueba aisladamente



3. Principio de Inversión de Dependencias

(DIP) Aplicado

Diagrama de Dependencias Correctas

REGLA MNEMOTÉCNICA

DEPENDENCIAS: →
Abstracciones
FLUJO: ←
Implementaciones

FLUJO DE CONTROL

APLICACIÓN

DOMINIO
necesita persistencia

INFRAESTRUCTURA
provee implementación

DEPENDENCIAS CORRECTAS

APLICACIÓN

IMPLEMENTACIÓN

INTERFAZ

Regla de Dependencias en Código

```
//  CORRECTO - Depende de abstracción
import type { EmployeeRepository } from '../domain/employee.repository';
// La aplicación SOLO conoce la interfaz

class EmployeeService {
  constructor(private repository: EmployeeRepository) {} // 

  async createEmployee(data: any) {
    // Usa repository.save(), repository.findById(), etc.
    // NO SABE si es memoria, MySQL o MongoDB
  }
}

//  INCORRECTO - Depende de implementación
import { InMemoryEmployeeRepository } from '../infrastructure/repositories/in-memory.repository';
// Ahora la aplicación ESTÁ ACOPLADA a memoria

class BadEmployeeService {
  constructor(private repository: InMemoryEmployeeRepository) {} // 
  // Cambiar a MySQL requiere modificar ESTA clase
}
```

4. Ventajas de esta Separación

Ventaja 1: Flexibilidad Tecnológica

```
// HOY: Desarrollo con memoria (rápido)
const repoDev = new InMemoryEmployeeRepository();

// MAÑANA: Producción con MySQL (robusto)
const repoProd = new MySQLEmployeeRepository(config);

// FUTURO: Migración a MongoDB (escalable)
const repoFuture = new MongoDBEmployeeRepository(config);

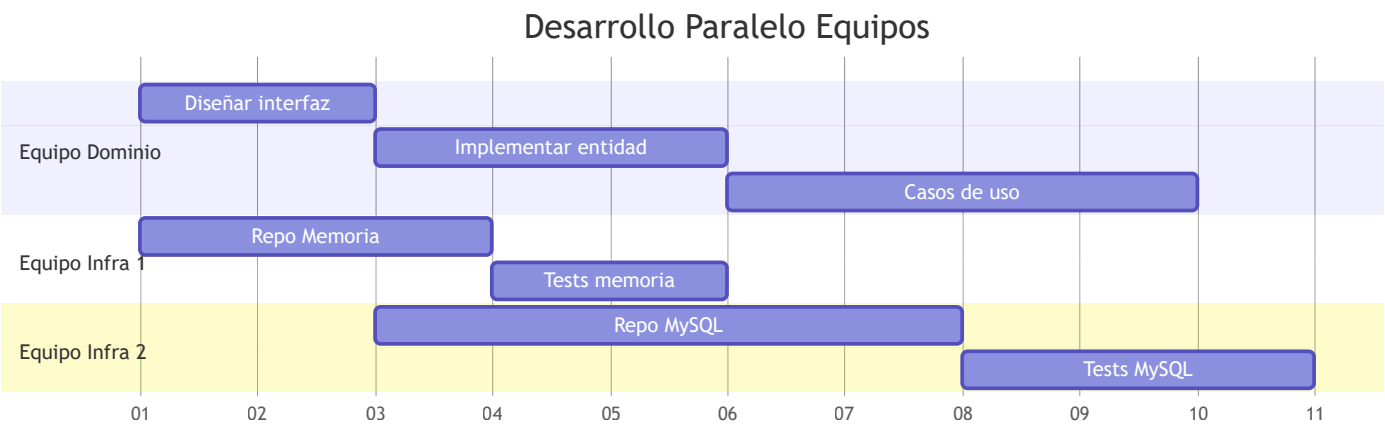
// ⚡ EL CÓDIGO DE NEGOCIO NO CAMBIA
const service = new EmployeeService(repoDev); // o repoProd, o repoFuture
```

Ventaja 2: Testing Efectivo

```
// Tests de Servicio (sin infraestructura real)
const mockRepo: EmployeeRepository = {
  save: jest.fn().mockResolvedValue(undefined),
  findById: jest.fn().mockResolvedValue(mockEmployee),
  // ... otros métodos mockeados
};

const service = new EmployeeService(mockRepo);
// ✅ Tests RÁPIDOS (ms) vs lentos con BD (segundos)
// ✅ AISLADOS (no fallan por problemas de BD)
// ✅ DETERMINISTAS (mismo resultado siempre)
```

Ventaja 3: Desarrollo en Paralelo



Ventaja 4: Deployment Flexible por Entorno

Entorno	Implementación	Beneficio
Local Dev	InMemoryRepository	Sin instalar BD, más rápido
CI/CD Tests	InMemoryRepository	Tests rápidos y aislados
Staging	MySQLRepository	Igual a producción
Producción	MySQLRepository + Cache	Performance óptimo

Ventaja 5: Mantenibilidad a Largo Plazo

```
// AÑO 1: Sistema nuevo
const repoV1 = new InMemoryEmployeeRepository();

// AÑO 2: Migración a BD
const repoV2 = new MySQLEmployeeRepository(config);

// AÑO 3: Agregamos cache
const repoV3 = new CachedRepository(
  new MySQLEmployeeRepository(config),
  new RedisCache()
);

// AÑO 4: Microservicios
const repoV4 = new ApiEmployeeRepository(apiClient);

// 🔄 EL CÓDIGO DEL NEGOCIO (2021) SIGUE FUNCIONANDO EN 2024
```

5. Pruebas del Repositorio

5.1 Archivo de Pruebas del Repositorio

```
// 📁 src/test-repository.ts
(async () => {
  console.log('🔧 PROBANDO REPOSITORIO EN MEMORIA');
  console.log('=====\\n');

  const { Employee } = await import('./domain/employee.entity.js');
  const { InMemoryEmployeeRepository } = await import('./infrastructure/repositories/in-memory.i

  const repo = new InMemoryEmployeeRepository();

  try {
    // 📌 PRUEBA CRUD COMPLETO
    console.log('1. 📄 CREATE - Guardar empleados');
    const emp1 = Employee.create('1111111111', 'Ana López', 2000);
    const emp2 = Employee.create('2222222222', 'Carlos Ruiz', 3500);
    await repo.save(emp1);
    await repo.save(emp2);
    console.log(`✅ Guardados: ${await repo.count()} empleados\\n`);

    console.log('2. 🔍 READ - Buscar por ID');
    const found = await repo.findById(emp1.getId());
    console.log(`✅ Encontrado: ${found?.getNombres()}\\n`);

    console.log('3. 🔍 READ - Buscar por cédula');
    const byCedula = await repo.findByCedula('2222222222');
    console.log(`✅ Por cédula: ${byCedula?.getNombres()}\\n`);

    console.log('4. 📄 READ - Obtener todos');
    const todos = await repo.findAll();
    console.log(`✅ Total: ${todos.length} empleados`);
    todos.forEach(e => console.log(`    • ${e.getNombres():} ${e.getSalario()}`));
    console.log('');

    console.log('5. ✏️ UPDATE - Actualizar salario');
    emp1.changeSalario(2500);
    await repo.update(emp1);
    const updated = await repo.findById(emp1.getId());
    console.log(`✅ Actualizado: ${updated?.getSalario()}\\n`);
```

```

console.log('6. 🗑 DELETE - Eliminar empleado');
const deleted = await repo.delete(emp2.getId());
console.log(`✅ Eliminado: ${deleted}, Quedan: ${await repo.count()}\n`);

console.log('7. 💰 READ - Rango de salarios ($2000-$3000)');
const enRango = await repo.findBySalaryRange(2000, 3000);
console.log(`✅ En rango: ${enRango.length} empleados\n`);

console.log('8. 🔍 EXISTS - Verificar existencia');
const existe = await repo.existsByCedula('1111111111');
console.log(`✅ ¿Existe 1111111111? ${existe ? 'Sí' : 'NO'}\n`);

console.log('🎉 ¡TODAS LAS PRUEBAS PASARON!');

} catch (error: any) {
  console.error('❌ ERROR:', error.message);
}
})();

```

5.2 Scripts para Ejecutar Pruebas

```

// 📁 package.json - Actualizar scripts
{
  "scripts": {
    "dev": "nodemon src/app.ts",
    "build": "tsc",
    "start": "node dist/app.js",
    "test:repository": "npx tsc && node dist/test-repository.js",
    "test:crud": "npx tsc && node dist/test-crud.js",
    "test:all": "npm run test:repository && npm run test:crud"
  }
}

```

```
# Ejecutar pruebas en PowerShell
```

```
npm run test:repository
```

```
# Salida esperada:
```

```
#  PROBANDO REPOSITORIO EN MEMORIA
```

```
# =====
```

```
#
```

```
# 1.  CREATE - Guardar empleados
```

```
#  Guardado en memoria: emp-1234567890-abc123
```

```
#  Guardado en memoria: emp-0987654321-def456
```

```
#  Guardados: 2 empleados
```

```
#
```

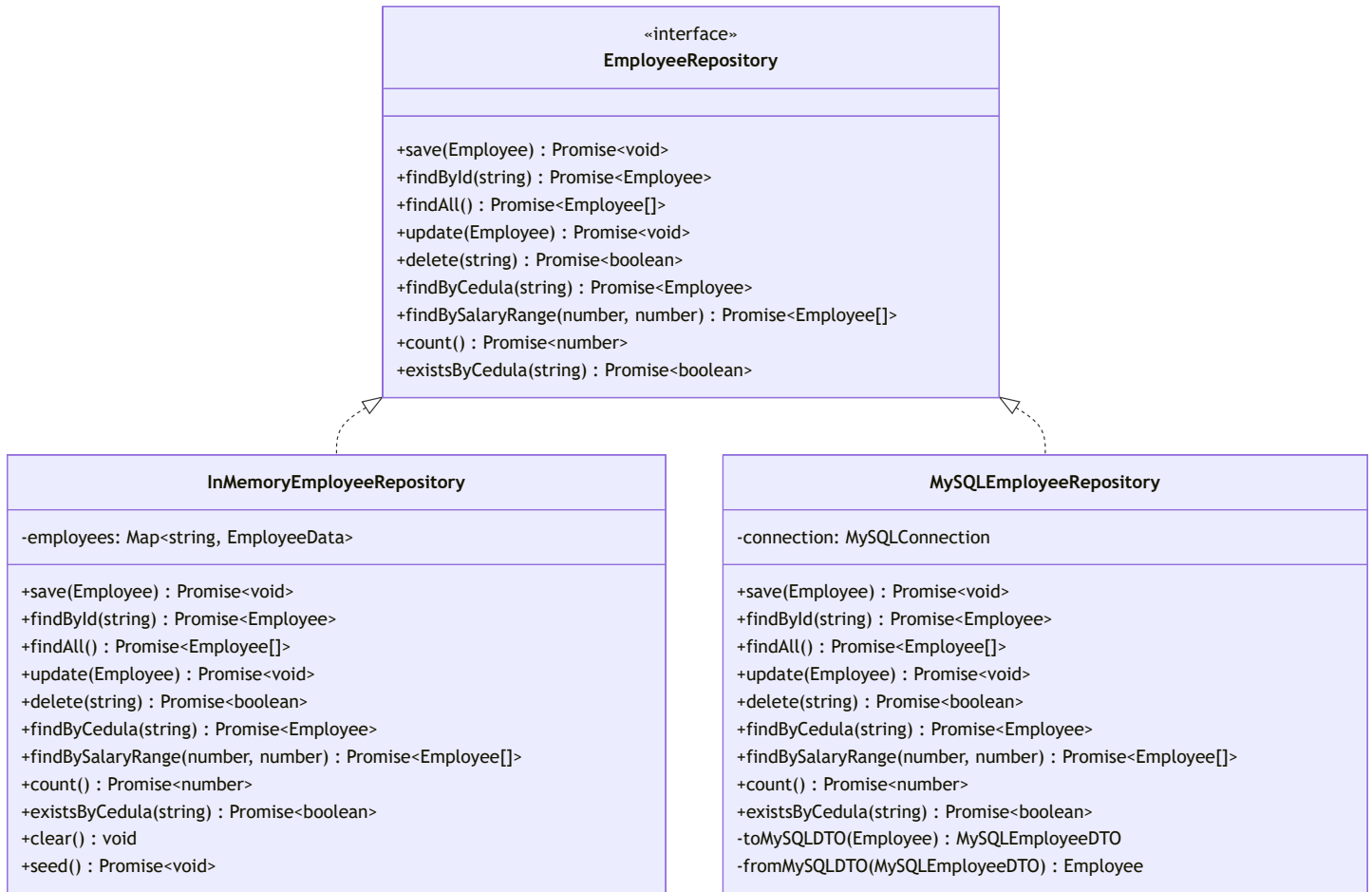
```
# ... (resto de pruebas)
```

```
#  ¡TODAS LAS PRUEBAS PASARON!
```



6. Conceptos Clave y Ejercicios

Diagrama de Clases del Patrón Repository



Ejercicio 1: Identificar Ubicación Correcta

// ¿DÓNDE VA CADA ARCHIVO? Clasifica:

// 1. employee.entity.ts ?

// RESPUESTA: DOMINIO ✓

// 2. mysql-employee.mapper.ts ?

// RESPUESTA: INFRAESTRUCTURA ✓

// 3. employee-events.ts ?

// RESPUESTA: DOMINIO ✓

// 4. redis-cache.decorator.ts ?

// RESPUESTA: INFRAESTRUCTURA ✓

// 5. salary-calculator.ts ?

// RESPUESTA: APLICACIÓN ✓

Ejercicio 2: Extender el Repositorio

```
// Agrega estos métodos:

// 1. En la INTERFAZ (dominio)
findByNombrePartial(nombre: string): Promise<Employee[]>;

// 2. En la IMPLEMENTACIÓN (infraestructura)
async findByNombrePartial(nombre: string): Promise<Employee[]> {
  // Búsqueda parcial insensible a mayúsculas
  const results: Employee[] = [];
  for (const data of this.employees.values()) {
    if (data.nombres.toLowerCase().includes(nombre.toLowerCase())) {
      results.push(Employee.fromData(data));
    }
  }
  return results;
}
```

Ejercicio 3: Implementar Otra Estrategia

```
// Crea FileSystemEmployeeRepository que:
// 1. Implemente EmployeeRepository
// 2. Guarde empleados en archivos JSON
// 3. Un archivo por empleado (id.json)
// 4. Carpeta `data/employees/` para almacenamiento

class FileSystemEmployeeRepository implements EmployeeRepository {
  private basePath = './data/employees/';

  async save(employee: Employee): Promise<void> {
    const path = `${this.basePath}${employee.getId()}.json`;
    const data = JSON.stringify(employee.toJSON(), null, 2);
    await fs.promises.writeFile(path, data);
  }

  // ... implementa otros métodos
}
```



7. Checklist de Completitud

Para el Estudiante:

- ☐ Entendí POR QUÉ la interfaz va en dominio y la implementación en infraestructura
- ☐ Implementé `EmployeeRepository` interface en `src/domain/`
- ☐ Implementé `InMemoryEmployeeRepository` en `src/infrastructure/repositories/`
- ☐ Usé `Map` para almacenamiento eficiente $O(1)$
- ☐ Implementé TODOS los métodos del contrato
- ☐ Creé archivo de pruebas `src/test-repository.ts`
- ☐ Ejecuté pruebas exitosamente con `npm run test:repository`
- ☐ Comprendí el Principio de Inversión de Dependencias

Para Evaluación del Profesor:

- ☐ **Interfaz en dominio:** Define contrato sin detalles técnicos
- ☐ **Implementación en infraestructura:** Contiene detalles (`Map`, `console.log`)
- ☐ **Métodos completos:** CRUD + búsquedas específicas
- ☐ **Pruebas exhaustivas:** Cobertura de todos los métodos
- ☐ **Separación correcta:** Ninguna dependencia incorrecta
- ☐ **Calidad de código:** Métodos claros, logging informativo

? 8. Preguntas de Reflexión

1. **¿Qué pasaría si pusieras la interfaz en infraestructura?**
La aplicación dependería de infraestructura, rompiendo DIP y acoplando las capas.
2. **¿Por qué los métodos devuelven Promises si estamos en memoria?**
Para mantener la misma firma que implementaciones asíncronas (MySQL, APIs).
3. **¿Cómo evitarías cédulas duplicadas en producción?**
El repositorio debería validar con `existsByCedula()` antes de `save()`.
4. **¿Qué ventaja tiene que `findByCedula()` sea $O(n)$ en memoria?**
En memoria es aceptable; en MySQL sería $O(1)$ con índice único.
5. **¿Cómo migrarías de memoria a MySQL sin downtime?**
Crear `MySQLRepository`, ejecutar ambos en paralelo, migrar datos gradualmente.



9. Próximos Pasos

Guía 4 - Capa de Aplicación (Servicios):

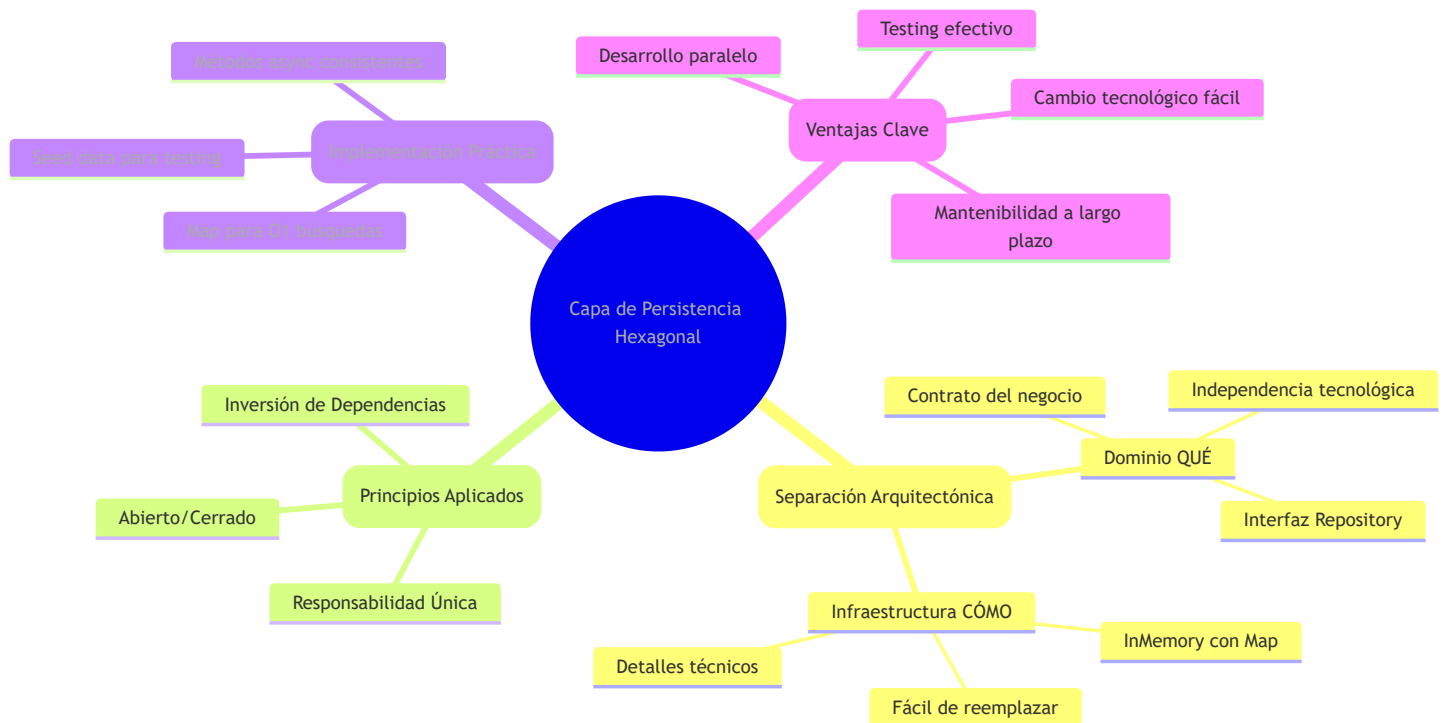
- Crear `EmployeeService` con casos de uso reales
- Implementar lógica de negocio que coordina entidad + repositorio
- Manejar validaciones y errores de aplicación
- Conectar TODO el flujo hexagonal

Tarea Preparatoria:

1. Ejecutar `npm run test:repository` y capturar salida
2. Pensar en 3 casos de uso reales para empleados (ej: aumentar salario masivo)
3. Investigar diferencias entre repositorio y servicio de aplicación



Resumen de lo Aprendido



✓ Has completado la Guía 3:

- ✓ Entendiste la separación dominio/infraestructura en persistencia
- ✓ Implementaste el patrón Repository con interfaz + implementación
- ✓ Aplicaste el Principio de Inversión de Dependencias correctamente

- ☒ Creaste un repositorio en memoria funcional con Map
- ☒ Probaste todas las operaciones CRUD exitosamente
- ☒ Comprendiste las ventajas de este diseño para el futuro